hw5

李毅PB22051031

1. 写出逆滤波和维纳滤波图象恢复的具体步骤。

逆滤波:

$g(x,y)=f(x,y)*h(x,y)+n(x,y)$

(1) 对$g(x,y)$和$h(x,y)$做傅里叶变换得到$G(u,v)$和$H(u,v)$

(2) 设计恢复转移函数$M(u,v)=1/H(u,v)$

(3) $\hat{F}(u,v)=G(u,v)*M(u,v)$，傅里叶逆变换得到$\hat{f}(u,v)$

改进:

$$M(u,v)=\begin{cases}\frac{1}{H(u,v)}, & \text{if } u^2+v^2<w_0^2\\ 1, & \text{else}\end{cases}$$

or:

$$M(u,v)=\begin{cases}k, & \text{if } H(u,v)<d\\ \frac{1}{H(u,v)}, & \text{else}\end{cases}$$

维纳滤波:

维纳滤波使得均方误差$e^2=E\{(f-\hat{f})^2\}$最小。

(1) 对$g(x,y)$和$h(x,y)$做傅里叶变换得到$G(u,v)$和$H(u,v)$

(2) 设计恢复转移函数$M(u,v)$

$$M(u,v)=\frac{1}{H(u,v)}*\frac{|H(u,v)|^2}{|H(u,v)|^2+K}$$

或:

$$M(u,v)=\frac{H^*(u,v)}{|H(u,v)|^2+K}$$

(3) $\hat{F}(u,v)=G(u,v)*M(u,v)$，傅里叶逆变换得到$\hat{f}(u,v)$

2. 推导水平匀速直线运动模糊的点扩展函数的数学公式并画出曲线。

水平匀速直线运动，可以记：$x_0(t)=ct/T$，$y_0(t)=0$。

$$g(x,y)=\int_0^T f(x-x_0(t),y-y_0(t))dt=\int_0^T f(x-ct/T,y)dt$$

由fourier变换平移性质，得:

$$G(u,v)=F(u,v)\int_0^T exp\{-j2\pi uct/T\}$$

$$H(u,v)=\int_0^T exp\{-j2\pi uct/T\}=Tsinc(uc)exp\{-j\pi uc\}$$

由fourier变换平移性质和sinc函数Fourier变换，得：

$$f(t)=\begin{cases}\frac{T}{c}, & 0<t<c\\ 0, & \text{else}\end{cases}$$

f(t)图像如下：

3. 编程实现lema.bmp的任意角旋转。

```
import math
import numpy as np
import cv2
import sys

def bilinear_interpolate(img, x, y):
    h = img.shape[0]
    w = img.shape[1]
    x0 = int(math.floor(x))
    x1 = x0 + 1
    y0 = int(math.floor(y))
    y1 = y0 + 1
    u = x - x0
    v = y - y0

    if x0 < 0 or x1 >= w or y0 < 0 or y1 >= h:
        if img.ndim == 3:
            return np.array([0, 0, 0], dtype=img.dtype)
        else:
            return 0

    I1 = img[y0, x0]
    I2 = img[y1, x0]
    I3 = img[y0, x1]
    I4 = img[y1, x1]

    return (1 - u) * (1 - v) * I1 + (1 - u) * v * I2 + u * (1 - v) * I3 + u * v *
I4
```

```python
def rotate_image(img, angle):
    angle_rad = math.radians(angle)
    cos_val = math.cos(angle_rad)
    sin_val = math.sin(angle_rad)

    h = img.shape[0]
    w = img.shape[1]

    #中心坐标，以中心建立坐标系
    cx, cy = w / 2, h / 2

    corners = np.array([
        [-cx, -cy],
        [w - cx, -cy],
        [-cx, h - cy],
        [w - cx, h - cy]
    ])

    # 旋转矩阵
    R = np.array([[cos_val, -sin_val],
                  [sin_val,  cos_val]])

    new_corners = np.dot(corners, R.T)
    x_coords = new_corners[:, 0] #旋转后的x坐标
    y_coords = new_corners[:, 1] #旋转后的y坐标

    # 旋转后尺寸发生变化，计算旋转后图像的宽高
    w_new = int(math.ceil(x_coords.max() - x_coords.min()))
    h_new = int(math.ceil(y_coords.max() - y_coords.min()))

    x_offset = -x_coords.min()
    y_offset = -y_coords.min()

    rotated = np.zeros((h_new, w_new), dtype=img.dtype)

    for i in range(h_new):
        for j in range(w_new):
            y_new = i - y_offset
            x_new = j - x_offset

            x_old =  cos_val * x_new + sin_val * y_new
            y_old = -sin_val * x_new + cos_val * y_new

            # 还原到原图坐标系
            x_src = x_old + cx
            y_src = y_old + cy

            if x_src >= 0 and x_src < w - 1 and y_src >= 0 and y_src < h - 1:
                rotated[i, j] = bilinear_interpolate(img, x_src, y_src)
            else:
                rotated[i, j] = 0 # 没有对应像素值时填充为0

    return rotated
```

```
angle = 60

img = cv2.imread('lena.bmp', 0)
rotated_np = rotate_image(img, angle)
output_filename = f"lema_rotated_{angle}.bmp"
cv2.imwrite(output_filename, np.uint8(rotated_np))
print(f"旋转后的图像已保存为 {output_filename}")
```

使用双线性插值法插值。结果如下：

旋转60度时：

旋转45度时：

旋转180度时：